

Software Testing, Selenium & Jira

Complete Interview Preparation Guide

From Fundamentals to Associate Software Engineer Level

Covers: Manual Testing · Smoke & Regression Testing

Selenium WebDriver · TestNG · POM · Jira

100+ Interview Questions (L1 & L2)

A self-study handbook for freshers and aspiring QA / SDET engineers

Table of Contents

#	Chapter
1	Introduction to Software Testing
2	SDLC and STLC
3	Levels and Types of Testing
4	Black-Box, White-Box and Gray-Box Testing
5	Test Case Design Techniques
6	Test Artifacts (Plan, Strategy, Case, Scenario)
7	Defect / Bug Life Cycle
8	Smoke Testing
9	Sanity Testing (and Smoke vs Sanity)
10	Regression Testing
11	Selenium - Introduction & Architecture
12	Selenium WebDriver - Locators
13	Selenium - Waits and Synchronization
14	Handling Web Elements
15	Alerts, Frames, Windows & Actions
16	TestNG Framework
17	Page Object Model (POM) & PageFactory
18	Data-Driven, Keyword-Driven & Hybrid Frameworks
19	Selenium Grid
20	Jira - Project Management for Testers
21	Agile / Scrum Basics for QA
22	L1 Interview Questions (Manual + Selenium + Jira)
23	L2 Interview Questions (Scenario / Coding)
24	Bonus: Resume Tips & Final Checklist

1. Introduction to Software Testing

What is Software Testing?

Software Testing is the process of evaluating a software application to find whether it meets the specified business and technical requirements and to identify defects (bugs) before the product is released to end users. It is a critical activity in the Software Development Life Cycle (SDLC) that ensures quality, reliability, and customer satisfaction.

Why is Testing Important?

- **Ensures Quality:** Verifies the software works as expected.
- **Saves Cost:** Bugs caught early are far cheaper to fix than bugs in production.
- **Improves Security:** Detects vulnerabilities before attackers do.
- **Boosts Customer Confidence:** A stable product builds trust and brand value.
- **Mandatory for Regulated Domains:** Healthcare, banking, aviation, etc. demand rigorous testing.
- **Enables Faster Release Cycles:** Automated tests support continuous delivery.

Seven Principles of Testing (ISTQB)

- 1. Testing shows the presence of defects** — Testing can prove the presence of bugs but cannot prove their absence.
- 2. Exhaustive testing is impossible** — Testing every input/combination is not feasible. Use risk-based and priority-based selection.
- 3. Early testing saves time and money** — Start testing activities (review, planning) as early as the requirement phase.
- 4. Defect clustering** — A small number of modules often contain the majority of defects (Pareto 80/20).
- 5. Pesticide paradox** — If the same tests are repeated, they stop finding new defects. Test cases must be reviewed and updated.
- 6. Testing is context-dependent** — Testing a banking app differs from testing a gaming app. Approach varies by domain.
- 7. Absence-of-errors fallacy** — A bug-free system that does not meet user needs is still useless.

Verification vs Validation

Aspect	Verification	Validation
Definition	Are we building the product right?	Are we building the right product?
Focus	Process, documents, design	Actual working product / behaviour
Methods	Reviews, walkthroughs, inspections	Unit, integration, system, UAT testing

Aspect	Verification	Validation
When	Done before development & during	Done after a build is ready
Type	Static testing	Dynamic testing

Quality Assurance vs Quality Control

Aspect	QA	QC
Goal	Prevent defects	Identify defects
Focus	Process-oriented	Product-oriented
Activities	Define standards, audits, training	Testing the software, reviewing results
Responsibility	Entire team	Specific testing team
Type	Proactive	Reactive

2. SDLC and STLC

Software Development Life Cycle (SDLC)

SDLC is a structured process used by software development teams to design, develop, test, and deploy high-quality software. Each phase has clear deliverables and roles.

- **1. Requirement Gathering & Analysis** — Business analysts collect requirements from stakeholders and prepare BRD/FRS.
- **2. Feasibility Study** — Technical, operational, and economic feasibility is evaluated.
- **3. Design** — Architects create HLD (High-Level Design) and LLD (Low-Level Design).
- **4. Coding / Implementation** — Developers convert design into source code.
- **5. Testing** — QA team verifies that the software meets requirements and is defect-free.
- **6. Deployment** — Software is released to the production environment.
- **7. Maintenance** — Post-release bug fixes, patches, and enhancements.

Common SDLC Models

Model	Key Idea	Best For
Waterfall	Sequential phases; one stage finishes before next begins	Small projects with clear, fixed requirements
V-Model	Each dev phase has a parallel testing phase	Projects needing strong verification (defence, medical)
Iterative	Build software in small repeated cycles	Large projects where requirements evolve slowly
Spiral	Iterative + heavy risk analysis at each cycle	High-risk, mission-critical projects
Agile / Scrum	Short sprints (2-4 weeks), frequent delivery, collaboration	Web/mobile products, evolving requirements
DevOps	Agile + CI/CD + automation across dev & ops	Cloud-native, frequently deployed products

Software Testing Life Cycle (STLC)

STLC is the testing-specific counterpart of SDLC. It defines a sequence of activities performed by the QA team to ensure quality goals are met.

- **1. Requirement Analysis** — QA studies requirements to identify what is testable. Activities: requirement traceability matrix (RTM) preparation, automation feasibility.
- **2. Test Planning** — Test Lead/Manager prepares Test Plan: scope, objective, schedule, resources, risks, tools, entry/exit criteria.
- **3. Test Case Design** — Write test cases, test scenarios, and test data. Review them with peers/stakeholders.
- **4. Test Environment Setup** — Configure servers, databases, browsers, test data. Smoke test to confirm environment is ready.

- **5. Test Execution** — Run test cases, log defects in tools like Jira, retest fixed defects, update RTM.
- **6. Test Closure** — Verify exit criteria, prepare test summary report, conduct retrospective meeting, archive test artifacts.

Entry and Exit Criteria (Important!)

Entry Criteria are the conditions that must be met before a phase can start. **Exit Criteria** are the conditions that must be met before a phase can end. For example, entry criteria of Test Execution phase: test cases reviewed, build deployed, environment ready. Exit criteria: all planned tests executed, no critical/high open defects, 95%% test cases passed.

3. Levels and Types of Testing

The Four Levels of Testing

Level	Performed By	Purpose
Unit Testing	Developer	Test the smallest piece of code (function/method) in isolation.
Integration Testing	Developer / QA	Verify interaction between two or more integrated modules.
System Testing	QA Team	Test the complete integrated system against requirements.
Acceptance Testing (UAT)	Client / End Users	Confirm the system satisfies business needs and is ready for release.

Integration Testing Approaches

- **Big Bang:** All modules are integrated and tested at once. Simple but hard to isolate defects.
- **Top-Down:** Test high-level modules first using stubs for missing low-level modules.
- **Bottom-Up:** Test low-level modules first using drivers to simulate higher modules.
- **Sandwich / Hybrid:** Combines top-down and bottom-up. Used in large layered systems.
- **Incremental:** Modules are integrated and tested one (or a few) at a time.

Functional vs Non-Functional Testing

Functional Testing	Non-Functional Testing
Verifies what the system does	Verifies how well the system performs
Based on functional requirements / use cases	Based on non-functional requirements (performance, security, etc.)
Examples: login, search, checkout, payment	Examples: load test, stress test, security test, usability test
Manual or automated using Selenium etc.	Tools: JMeter, LoadRunner, OWASP ZAP, Burp Suite

Common Types of Testing

- **Smoke Testing** — Quick check that critical functionality works on a new build.
- **Sanity Testing** — Narrow regression after minor changes or bug fixes.
- **Regression Testing** — Re-run existing tests after a change to ensure nothing is broken.
- **Re-testing** — Run the exact failed test again after the bug is fixed.
- **Functional Testing** — Test individual business functions.
- **UI / GUI Testing** — Verify layouts, colors, navigation, responsiveness.
- **Usability Testing** — Evaluate how user-friendly the application is.
- **Compatibility Testing** — Run on multiple browsers, OSes, devices, resolutions.
- **Performance Testing** — Load, stress, spike, endurance testing of the system.

- **Security Testing** — Penetration test, vulnerability scan, authorisation/authentication checks.
- **Acceptance Testing** — UAT, alpha (internal), beta (limited external users).
- **Exploratory Testing** — Simultaneous learning, designing and executing of tests (no fixed scripts).
- **Ad-hoc Testing** — Informal testing without any plan; relies on tester's intuition.
- **Monkey / Random Testing** — Provide random inputs to find crashes.
- **Localisation & Internationalisation (L10N / I18N)** — Test for languages, currency, date formats.
- **Accessibility Testing** — Make sure the app works for users with disabilities (WCAG).

4. Black-Box, White-Box and Gray-Box Testing

Aspect	Black-Box	White-Box	Gray-Box
Internal code knowledge	No	Yes	Partial
Performed by	Testers / QA	Developers	Testers with code awareness
Focus	Inputs & outputs	Code paths, branches	Mix of UI and code logic
Techniques	BVA, ECP, decision tables	Statement, branch, path, loop coverage	Input coverage, regression
Examples	Functional, UAT, system	Unit, code coverage	Integration, penetration

White-Box Testing Techniques

- **Statement coverage:** Execute every line of code at least once.
- **Branch coverage:** Execute each branch (if/else) at least once.
- **Path coverage:** Execute every possible path from start to end.
- **Condition coverage:** Each Boolean sub-expression must evaluate to both true and false.
- **Loop coverage:** Execute loops zero times, once, and many times.

5. Test Case Design Techniques

Test design techniques help generate a minimum set of test cases that still provide maximum coverage. They are the toolkit every tester must master.

5.1 Equivalence Class Partitioning (ECP)

Divide the input domain into classes such that all values in a class are expected to behave the same. Pick one value from each class. **Example:** For an age field that accepts 18-60:

- Invalid low: -10, 0, 17 → one test (say 10)
- Valid: 18 to 60 → one test (say 30)
- Invalid high: 61, 100 → one test (say 70)

5.2 Boundary Value Analysis (BVA)

Defects often occur at edges. For a range [min, max], test min-1, min, min+1, max-1, max, max+1. **Example:** for age 18-60 → test 17, 18, 19, 59, 60, 61.

5.3 Decision Table Testing

Capture combinations of conditions and the corresponding actions. Useful for business rules with many flags.

Cond/Action	Rule 1	Rule 2	Rule 3	Rule 4
Valid user	T	T	F	F
Valid password	T	F	T	F
Action: Login	Yes	No	No	No
Action: Error	-	Yes	Yes	Yes

5.4 State Transition Testing

Model the system as states and transitions. Test valid and invalid transitions. **Example:** ATM card states — Idle → Active → Blocked. Three wrong PINs trigger Active → Blocked.

5.5 Use Case Testing

Derive test cases directly from user use cases (login, place order, etc.). Excellent for capturing end-to-end business flows.

5.6 Error Guessing

Experience-based technique. Tester imagines where developers commonly make mistakes: empty fields, special characters, max length, duplicate submissions, network drop mid-transaction, etc.

6. Test Artifacts

Test Plan

A high-level document prepared by the Test Manager/Lead that defines the testing approach for a project. Major sections:

- Test plan ID, version, author
- Objectives and scope (what is in / out)
- Items to be tested and not tested
- Test approach & deliverables
- Roles & responsibilities
- Schedule, milestones, resources
- Entry & exit criteria
- Risks & mitigations
- Tools & environments

Test Strategy

An organisation-level document (one per company/department) describing *how* testing is approached for all projects: standards, tools, automation policy, defect management process, etc.

Test Scenario vs Test Case

Test Scenario	Test Case
High-level description of what to test	Detailed steps to verify a scenario
Example: "Verify login"	Step 1: Open URL. Step 2: Enter user. Step 3: Enter password...
Less time to write, less detailed	More time, fully documented
1 scenario can have many test cases	Belongs to a single scenario

Sample Test Case Format

Field	Description
Test Case ID	TC_LOGIN_001
Title	Verify successful login with valid credentials
Pre-conditions	User is registered and on the login page
Test Data	username = user1@test.com / password = Pass@123
Steps	1. Open https://app.example.com 2. Enter username 3. Enter password 4. Click

Field	Description
Expected Result	User is redirected to the dashboard page
Actual Result	(filled after execution)
Status	Pass / Fail / Blocked
Priority	High
Severity	Critical

Requirement Traceability Matrix (RTM)

A table that maps each requirement to the test cases that verify it. Ensures 100%% requirement coverage and helps with impact analysis when a requirement changes.

Req ID	Requirement	Test Case IDs	Status
REQ-01	User can log in with email	TC_LOGIN_001, TC_LOGIN_002	Pass
REQ-02	User can reset forgotten password	TC_PWD_001, TC_PWD_002	Pass
REQ-03	Account locks after 3 failed attempts	TC_LOGIN_005	Fail

Severity vs Priority (must know!)

Severity	Priority
Impact of the defect on the software	Urgency to fix the defect from a business view
Decided by tester / QA	Decided by product owner / business
Levels: Critical, Major, Minor, Trivial	Levels: High, Medium, Low
Example: App crash on launch & rarr; Critical severity	Example: Wrong logo on home page & rarr; High priority but Low severity

Real example combinations

- **High Severity, High Priority:** Login completely fails for everyone.
- **High Severity, Low Priority:** Crash on a rarely-used admin screen near release.
- **Low Severity, High Priority:** Spelling mistake in the company logo on the home page.
- **Low Severity, Low Priority:** Footer text alignment is slightly off.

7. Defect / Bug Life Cycle

A defect goes through a series of states from discovery to closure. Understanding this flow is essential for working effectively with Jira.

New: Tester logs a new defect.

Assigned: Test Lead/Manager assigns it to a developer.

Open: Developer is analysing/fixing the defect.

Fixed: Developer has fixed the issue and asks QA to verify.

Retest: Tester re-runs the failed test on the new build.

Verified: Tester confirms the defect is fixed.

Closed: The defect is officially closed.

Reopened: If still reproducible, defect is reopened.

Rejected: Developer believes it is not a defect (e.g., works as designed).

Deferred: Valid defect but will be fixed in a later release.

Duplicate: The defect is the same as an existing one.

Typical Bug Report Fields (Jira)

- Bug ID, Summary, Description
- Steps to reproduce (precise!)
- Expected vs Actual result
- Severity, Priority
- Environment (OS, browser, build no.)
- Screenshots / video / logs
- Reported by / Assigned to
- Found in build / Fixed in build
- Status, Labels, Components, Sprint

8. Smoke Testing

Definition

Smoke testing is a preliminary, shallow but wide test executed on a new software build to confirm that the most important functionalities are working and that the build is stable enough to proceed with detailed testing. It is also called **Build Verification Testing (BVT)**.

The name comes from hardware testing — when an engineer powered on a newly assembled circuit, if no smoke appeared, the board was good enough to test further.

Characteristics

- Performed on every new build before detailed testing
- Covers only critical and end-to-end happy paths
- Quick to execute (usually 30 minutes to a couple of hours)
- Performed by QA or sometimes by developers
- Can be manual or automated (Selenium, Cypress, CI pipelines)
- Acts as the entry criterion for the rest of QA

When to perform Smoke Testing?

- When a new build is delivered to QA
- Before starting any regression or functional test cycle
- After every deployment to a new environment (QA, staging, production)
- After major code merges in CI/CD pipelines

Sample Smoke Test Cases for an E-commerce Site

- Application launches without errors
- User is able to log in with valid credentials
- Home page loads with banners and categories
- Product search returns results
- Add to cart works
- Checkout page loads with order summary
- User can log out successfully

Benefits

- Detects show-stopper issues very early
- Saves QA time — no point running 1000 cases on a broken build
- Improves communication: QA can quickly reject an unstable build

- Boosts confidence in CI/CD pipelines when automated

Automating Smoke Tests with Selenium - Sample

```
@Test
public void smokeTest_homePageLoads() {
    driver.get("https://example.com");
    Assert.assertEquals(driver.getTitle(), "Welcome");
    Assert.assertTrue(driver.findElement(By.id("logo")).isDisplayed());
}

@Test
public void smokeTest_loginWorks() {
    driver.findElement(By.id("user")).sendKeys("u1");
    driver.findElement(By.id("pass")).sendKeys("p1");
    driver.findElement(By.id("loginBtn")).click();
    Assert.assertTrue(driver.getCurrentUrl().contains("dashboard"));
}
```

9. Sanity Testing & Smoke vs Sanity

Definition

Sanity testing is a narrow and deep test performed on a build that has received minor changes or bug fixes. It verifies only the affected area to confirm the fix worked and did not break related functionality. Sanity is essentially a small, targeted subset of regression.

Smoke vs Sanity Testing (very common interview question!)

Aspect	Smoke	Sanity
Performed on	Every new initial build	A relatively stable build with minor changes
Coverage	Wide but shallow	Narrow but deep
Goal	Build acceptance for testing	Verify a specific fix or change
Documentation	Usually scripted/automated	Often unscripted (ad-hoc)
Subset of	Acceptance / system test	Regression testing
Performed by	QA or Dev	QA

10. Regression Testing

Definition

Regression testing is the re-execution of a set of test cases on a modified application to ensure that existing functionality still works after a code change — whether the change is a new feature, enhancement, configuration update, or bug fix.

When to perform?

- After every code change (CI pipelines run regression on each commit)
- After a bug fix — ensure the fix did not break anything else
- After integration of a new module
- Before release as a final go/no-go check
- After environment / configuration changes

Types of Regression Testing

- **Unit Regression:** Re-run unit tests on the changed module only, in isolation.
- **Partial Regression:** Re-run tests on the changed module plus its dependent modules.
- **Complete (Full) Regression:** Re-run the entire suite when changes are widespread.
- **Selective Regression:** Choose only the most relevant subset of test cases.
- **Progressive Regression:** Run new test cases written for new functionality alongside existing ones.
- **Corrective Regression:** No requirements changed; re-use existing test cases.
- **Retest-All Regression:** Re-run all test cases from scratch (expensive).

Regression Test Selection Strategy

- Frequently used and business-critical features
- Test cases that historically caught defects (defect clustering)
- Modules touched by the latest code change (impact analysis)
- Boundary value and complex flow test cases
- Test cases for previously fixed defects (avoid bug recurrence)

Why Automate Regression?

Regression suites grow with each release. Running 500+ tests manually after every code change is impractical. Automation (Selenium + TestNG, Cypress, Playwright) makes regression repeatable, fast, reliable, and suitable for CI/CD.

Common Tools for Regression Automation

- **Selenium WebDriver** — web automation
- **TestNG / JUnit** — test orchestration
- **Cypress / Playwright** — modern JS-based
- **Appium** — mobile
- **Jenkins / GitHub Actions / GitLab CI** — CI scheduling
- **JIRA + Xray / Zephyr** — test management

Retesting vs Regression Testing

Retesting	Regression Testing
Run a previously failed test to verify a fix	Run other tests to ensure no new defects appeared
Planned only for failed tests	Planned for the whole impacted area
Cannot be automated easily (depends on fix)	Heavily automated
Has higher priority than regression	Done after retesting

11. Selenium - Introduction & Architecture

What is Selenium?

Selenium is an open-source suite of tools used for automating web browsers. It supports multiple browsers (Chrome, Firefox, Edge, Safari) and multiple languages (Java, Python, C#, Ruby, JavaScript, Kotlin). It is the de-facto standard for web UI automation worldwide.

Selenium Suite Components

- **Selenium IDE** — A browser plugin (Chrome/Firefox) for recording and playback. Best for beginners and quick exploratory automation.
- **Selenium RC (deprecated)** — The original remote-control tool, replaced by WebDriver in 2016 (Selenium 3).
- **Selenium WebDriver** — The core API for programmatic browser automation. The most widely used component.
- **Selenium Grid** — Runs tests in parallel across many browsers/machines, supporting cross-browser and distributed testing.

Selenium WebDriver Architecture

Selenium WebDriver follows a client-server architecture using the W3C WebDriver protocol. The flow is:

- **Test Script (Client):** Written in Java/Python/etc. using Selenium client libraries.
- **JSON Wire Protocol (now W3C protocol):** Test commands are sent as HTTP requests in JSON.
- **Browser Driver:** Each browser has its own driver (ChromeDriver, GeckoDriver, EdgeDriver, SafariDriver).
- **Browser:** The driver translates commands into native browser actions.

From Selenium 4 onwards, the JSON Wire Protocol is replaced by the **W3C WebDriver Standard**, which makes communication faster, language-neutral, and consistent across browsers.

Why Selenium is Popular

- Open source & free
- Cross-browser and cross-platform
- Multi-language support
- Huge community & documentation
- Integrates with TestNG, JUnit, Cucumber, Maven, Jenkins, Docker, etc.
- Supports headless execution for CI

Selenium's Limitations

- Only automates browsers (cannot test desktop or native mobile apps directly)
- No built-in reporting — need TestNG/Extent Reports/Allure
- Cannot handle captcha / OTP
- Cannot read graphical objects like bar codes or images (without OCR)
- Performance/load testing is not its purpose — use JMeter

First Selenium Java Program

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class FirstTest {
    public static void main(String[] args) {
        WebDriver driver = new ChromeDriver();
        driver.get("https://www.google.com");
        System.out.println(driver.getTitle());
        driver.findElement(By.name("q")).sendKeys("Selenium");
        driver.quit();
    }
}
```

12. Selenium WebDriver - Locators

Locators are how Selenium finds elements on a web page. Choosing the right locator is the single most important skill for writing stable Selenium tests.

The 8 Standard Locators

Locator	Java Syntax	When to use
id	<code>By.id("email")</code>	Best - usually unique and fast
name	<code>By.name("username")</code>	Used for form inputs
className	<code>By.className("btn-primary")</code>	When class is unique
tagName	<code>By.tagName("a")</code>	Get a list of similar tags (links, inputs)
linkText	<code>By.linkText("Sign In")</code>	Only for anchor (<a>) elements - exact match
partialLinkText	<code>By.partialLinkText("Sign")</code>	When link text is dynamic
css selector	<code>By.cssSelector("input#email")</code>	Powerful, fast, recommended
xpath	<code>By.xpath("//input[@id='email']")</code>	Most flexible (forward + backward navigation)

CSS Selector Cheat Sheet

Pattern	Meaning
<code>#email</code>	Element with id email
<code>.btn</code>	Element with class btn
<code>input[type='text']</code>	Input tag with type=text
<code>form > input</code>	Direct child input of form
<code>form input</code>	Any descendant input of form
<code>input[name^='user']</code>	name attribute starts with 'user'
<code>input[name\$='name']</code>	name attribute ends with 'name'
<code>input[name*='user']</code>	name attribute contains 'user'
<code>li:nth-child(2)</code>	Second li in its parent
<code>li:first-child</code>	First li in its parent

XPath Cheat Sheet

Pattern	Meaning
//input[@id='email']	Input anywhere with id=email
//input[@name='user' and @type='text']	Multiple conditions
//a[text()='Login']	Anchor with exact text 'Login'
//a[contains(text(),'Log')]	Anchor whose text contains Log
//div[contains(@class,'header')]	Div with class containing header
//input[starts-with(@id,'user')]	id starts with user
//input[@id='email']/parent::div	Parent of the input
//div[@id='form']/descendant::input	All input descendants
(//button)[2]	Second button on the page
//tr[td[text()='John']]	Row containing td with text John

Absolute vs Relative XPath

- **Absolute XPath:** /html/body/div[1]/form/input[2] — starts at root. Very fragile, breaks on any change.
- **Relative XPath:** //input[@id='email'] — starts with // and searches anywhere. Always preferred.

CSS vs XPath

CSS	XPath
Faster in most browsers	Slightly slower
Forward direction only	Forward and backward (parent::, ancestor::)
Cannot find element by text	Can find using text() and contains(text(),")
Simpler, cleaner syntax	More powerful but verbose

Selenium 4 - New Relative Locators

```
import static org.openqa.selenium.support.locators.RelativeLocator.with;

WebElement password = driver.findElement(
    with(By.tagName("input")).below(emailField));

WebElement label = driver.findElement(
    with(By.tagName("label")).toLeftOf(checkBox));

// near(), above(), below(), toLeftOf(), toRightOf() are available
```

13. Selenium - Waits and Synchronization

Web pages load asynchronously. Without waits, Selenium throws *NoSuchElementException* or *ElementNotInteractableException*. Three types of waits help synchronize tests with the application.

13.1 Implicit Wait

Applies globally to all findElement calls. Selenium polls the DOM for the specified time before giving up.

```
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

- **Pros:** Easy — set once.
- **Cons:** Cannot wait for conditions other than presence; slows scripts down.

13.2 Explicit Wait (WebDriverWait)

Wait until a specific condition is true on a specific element. The recommended approach.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
WebElement btn = wait.until(
    ExpectedConditions.elementToBeClickable(By.id("submit")));
btn.click();
```

Common ExpectedConditions:

- `visibilityOfElementLocated(by)`
- `presenceOfElementLocated(by)`
- `elementToBeClickable(by)`
- `textToBePresentInElement(elem, text)`
- `alertIsPresent()`
- `titleContains(text)`
- `frameToBeAvailableAndSwitchToIt(by)`
- `invisibilityOfElementLocated(by)`

13.3 Fluent Wait

Like an explicit wait but you can configure the polling interval and ignored exceptions.

```
Wait<WebDriver> wait = new FluentWait<>(driver)
    .withTimeout(Duration.ofSeconds(30))
    .pollingEvery(Duration.ofSeconds(2))
    .ignoring(NoSuchElementException.class);

WebElement el = wait.until(d -> d.findElement(By.id("ok")));
```

Best Practices

- Avoid `Thread.sleep()` — it is the laziest and most fragile option.
- Prefer explicit waits over implicit; do not mix the two.

- Set explicit waits per element/condition, not globally.
- Wrap repeated waits in utility methods (e.g., `waitForClickable`).

14. Handling Web Elements

14.1 Text Boxes

```
WebElement email = driver.findElement(By.id("email"));
email.clear();
email.sendKeys("user@test.com");
String value = email.getAttribute("value");
```

14.2 Buttons & Links

```
driver.findElement(By.id("loginBtn")).click();
driver.findElement(By.linkText("Forgot password?")).click();
```

14.3 Checkboxes

```
WebElement chk = driver.findElement(By.id("terms"));
if (!chk.isSelected()) {
    chk.click();
}
```

14.4 Radio Buttons

```
List<WebElement> radios = driver.findElements(By.name("gender"));
for (WebElement r : radios) {
    if (r.getAttribute("value").equals("Male")) {
        r.click();
        break;
    }
}
```

14.5 Dropdowns (Select tag)

```
import org.openqa.selenium.support.ui.Select;

Select country = new Select(driver.findElement(By.id("country")));
country.selectByVisibleText("India");
country.selectByValue("IN");
country.selectByIndex(2);

// For multi-select
country.deselectAll();
```

Note: The Select class only works for native <select> tags. Custom dropdowns built with <div>+ need manual click + option selection.

14.6 Tables

```
List<WebElement> rows = driver.findElements(
    By.xpath("//table[@id='users']/tbody/tr"));
for (WebElement row : rows) {
    List<WebElement> cells = row.findElements(By.tagName("td"));
    System.out.println(cells.get(0).getText() + " | "
        + cells.get(1).getText());
}
```

15. Alerts, Frames, Windows & Actions

15.1 JavaScript Alerts

```
Alert alert = driver.switchTo().alert();
System.out.println(alert.getText());
alert.accept();      // OK
alert.dismiss();     // Cancel
alert.sendKeys("hi"); // prompt-type alerts only
```

15.2 Frames / iFrames

```
driver.switchTo().frame("frameName");
driver.switchTo().frame(0);           // by index
driver.switchTo().frame(driver.findElement(By.id("frm1")));

// Back to main page
driver.switchTo().defaultContent();
driver.switchTo().parentFrame();
```

15.3 Multiple Windows / Tabs

```
String parent = driver.getWindowHandle();
Set<String> all = driver.getWindowHandles();

for (String handle : all) {
    if (!handle.equals(parent)) {
        driver.switchTo().window(handle);
        // do something in child
        driver.close();
    }
}
driver.switchTo().window(parent);
```

15.4 Actions Class (mouse, keyboard, drag-drop)

```
Actions act = new Actions(driver);

// hover
act.moveToElement(menu).perform();

// right click
act.contextClick(elem).perform();

// double click
act.doubleClick(elem).perform();

// drag & drop
act.dragAndDrop(source, target).perform();

// keyboard
act.sendKeys(Keys.ENTER).perform();
act.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform();
```

15.5 File Upload

```
// HTML must be <input type="file"/>
driver.findElement(By.id("upload"))
    .sendKeys("C:\\files\\photo.png");
```

15.6 JavaScript Executor

```
JavascriptExecutor js = (JavascriptExecutor) driver;

// scroll
js.executeScript("window.scrollTo(0,500)");
js.executeScript("arguments[0].scrollIntoView(true)", elem);

// click when normal click fails
js.executeScript("arguments[0].click()", elem);

// fetch page title
String title = (String) js.executeScript("return document.title");
```

15.7 Screenshots

```
File src = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
FileUtils.copyFile(src, new File("./reports/login.png"));
```

16. TestNG Framework

TestNG (Test Next Generation) is the most popular test orchestration framework used with Selenium and Java. It provides annotations, assertions, parallel execution, reports, and data-driven testing.

Important Annotations

Annotation	Purpose
@BeforeSuite	Runs once before all tests in the suite
@BeforeTest	Runs before each <test> tag in testng.xml
@BeforeClass	Runs before the first method of the class
@BeforeMethod	Runs before each @Test method
@Test	Marks a method as a test
@AfterMethod	Runs after each test method
@AfterClass	Runs after all test methods in the class
@AfterTest	Runs after each <test> tag
@AfterSuite	Runs once after the suite finishes
@DataProvider	Supplies parameters to tests (data-driven)
@Parameters	Pass parameters from testng.xml
@Listeners	Register custom listeners (e.g., for reporting)

@Test attributes

- **priority** — order of execution (lower runs first).
- **dependsOnMethods** — run only if a given method passed.
- **enabled = false** — skip the test temporarily.
- **invocationCount = n** — run the test n times.
- **timeOut = ms** — fail if test exceeds time limit.
- **groups = {"smoke","reg"}** — tag tests to be selected by group.
- **dataProvider = "name"** — link to a @DataProvider.

Sample TestNG Test

```
public class LoginTest {
    WebDriver driver;

    @BeforeMethod
    public void setup() {
```

```

        driver = new ChromeDriver();
        driver.get("https://example.com");
    }

    @Test(priority = 1, groups = {"smoke"})
    public void validLogin() {
        driver.findElement(By.id("u")).sendKeys("u1");
        driver.findElement(By.id("p")).sendKeys("p1");
        driver.findElement(By.id("btn")).click();
        Assert.assertEquals(driver.getTitle(), "Dashboard");
    }

    @AfterMethod
    public void tearDown() { driver.quit(); }
}

```

testng.xml - controlling execution

```

<suite name="Suite1" parallel="tests" thread-count="3">
  <test name="ChromeTests">
    <parameter name="browser" value="chrome"/>
    <classes>
      <class name="tests.LoginTest"/>
      <class name="tests.SearchTest"/>
    </classes>
  </test>
</suite>

```

DataProvider (data-driven)

```

@DataProvider(name="users")
public Object[][] getUsers() {
    return new Object[][] {
        {"u1", "p1"},
        {"u2", "p2"},
        {"u3", "p3"}
    };
}

@Test(dataProvider="users")
public void login(String user, String pass) { ... }

```

Assertions

- **Hard Assertion (Assert):** Stops execution on failure. `Assert.assertEquals(actual, expected)`
- **Soft Assertion (SoftAssert):** Continues execution; call `softAssert.assertAll()` at the end to report all failures.

17. Page Object Model (POM) & PageFactory

POM is a design pattern in Selenium where every web page is represented by a Java class. Locators and page actions live in that class; test scripts only call methods. This makes tests readable and maintainable.

Benefits of POM

- Locator changes require update in one place only
- Re-usable methods across tests
- Cleaner, more readable test code
- Easier maintenance for large test suites

Basic POM Structure

```
public class LoginPage {
    WebDriver driver;

    By userField = By.id("u");
    By passField = By.id("p");
    By loginBtn = By.id("btn");

    public LoginPage(WebDriver driver) { this.driver = driver; }

    public void enterUser(String u) {
        driver.findElement(userField).sendKeys(u);
    }
    public void enterPass(String p) {
        driver.findElement(passField).sendKeys(p);
    }
    public DashboardPage clickLogin() {
        driver.findElement(loginBtn).click();
        return new DashboardPage(driver);
    }
    public DashboardPage loginAs(String u, String p) {
        enterUser(u); enterPass(p); return clickLogin();
    }
}
```

PageFactory

Selenium's PageFactory is an inbuilt helper that initialises page elements with @FindBy annotations — reducing boilerplate.

```
public class LoginPage {
    @FindBy(id="u") WebElement userField;
    @FindBy(id="p") WebElement passField;
    @FindBy(id="btn") WebElement loginBtn;

    public LoginPage(WebDriver driver) {
        PageFactory.initElements(driver, this);
    }
    public void loginAs(String u, String p) {
```

```
        userField.sendKeys(u);
        passField.sendKeys(p);
        loginBtn.click();
    }
}
```

@FindBy locator variants

```
@FindBy(id="email")
@FindBy(name="email")
@FindBy(className="form-control")
@FindBy(xpath="//input[@id='email']")
@FindBy(css="input#email")
@FindBys({ @FindBy(id="form"), @FindBy(tagName="input") })
@FindAll({ @FindBy(id="a"), @FindBy(id="b") })
```

18. Data-Driven, Keyword-Driven & Hybrid Frameworks

Framework Comparison

Framework	Idea	Pros	Cons
Linear	Record & playback, one big script	Easy for beginners	Hard to maintain
Modular	Break app into modules with reusable functions	Reusability	Still test-data is hard-coded
Data-Driven	Test data lives in external file (Excel, CSV, JSON, DB)	Separates test data from test logic	Setup effort
Keyword-Driven	Actions written as keywords (login, click, etc.) at user level	No technical knowledge required	Framework heavy
Hybrid	Combines POM + data-driven + keyword-driven	Most flexible, reusable	Complex to build
BDD	Behaviour-Driven Development using Gherkin	Readable specs in plain English	Extra layer of abstraction

Reading Excel with Apache POI (Data-Driven)

```

FileInputStream fis = new FileInputStream("data.xlsx");
XSSFWorkbook wb = new XSSFWorkbook(fis);
XSSFSheet sheet = wb.getSheet("Login");
int rows = sheet.getLastRowNum();
for (int i = 1; i <= rows; i++) {
    String user = sheet.getRow(i).getCell(0).getStringCellValue();
    String pass = sheet.getRow(i).getCell(1).getStringCellValue();
    // call test
}
wb.close();

```

Sample Maven Project Structure

```

src/
  main/
    java/
      pages/      (POM classes)
      utils/      (Excel, Config, ScreenshotUtil)
      base/       (BaseTest with driver setup)
  test/
    java/
      tests/      (TestNG test classes)
  resources/
    testdata/    (xlsx, json)
    config.properties
pom.xml
testng.xml

```


19. Selenium Grid

Selenium Grid lets you run tests on different machines, browsers, and OS combinations in parallel. It is essential for cross-browser testing and faster execution.

Architecture (Selenium 4 - new Grid)

- **Router:** Single entry point for tests.
- **Session Map:** Maps sessions to nodes.
- **Distributor:** Decides which node runs the test.
- **Event Bus:** Inter-component messaging.
- **Node(s):** Actual machines/containers running the browser.

Starting Grid (Selenium 4)

```
# Standalone mode
java -jar selenium-server-<version>.jar standalone

# Distributed (hub + node)
java -jar selenium-server-<version>.jar hub
java -jar selenium-server-<version>.jar node --hub http://hub:4444
```

Using RemoteWebDriver in test

```
ChromeOptions opt = new ChromeOptions();
opt.setPlatformName("Windows 10");

WebDriver driver = new RemoteWebDriver(
    new URL("http://localhost:4444/wd/hub"), opt);
```

20. Jira - Project Management for Testers

What is Jira?

Jira is a powerful tool by Atlassian used to plan, track, and manage software projects. It is the industry standard for Agile teams, bug tracking, and test management.

Key Features

- Issue / bug tracking
- Scrum and Kanban boards
- Backlog, sprints, releases (versions)
- JQL (Jira Query Language)
- Custom workflows
- Dashboards, reports, gadgets
- Integration with Bitbucket, GitHub, Jenkins, Slack, Confluence
- Add-ons such as Xray, Zephyr for test management

Common Issue Types

Type	Purpose
Epic	Large body of work spanning multiple sprints
Story	User story representing a feature from user perspective
Task	General work item
Sub-task	Smaller piece of work under a story/task
Bug	Defect found in the application
Improvement	Enhancement to existing feature
New Feature	Brand-new functionality

Typical Jira Workflow for a Bug

Open → In Progress → Fixed → In Testing → Verified → Closed (or Reopened)

Fields in a Jira Bug Ticket

- Project, Issue Type
- Summary, Description (with reproduction steps)
- Assignee, Reporter
- Priority (Highest, High, Medium, Low, Lowest)

- Severity (custom field in many projects)
- Labels, Components, Fix Version, Affects Version
- Sprint, Story Points
- Attachments (screenshots, logs, videos)
- Linked Issues (blocks, is blocked by, duplicates, relates to)

JQL - Jira Query Language

Examples of useful JQL queries:

```
project = "PORTAL" AND status = "Open"

assignee = currentUser() AND resolution = Unresolved

issuetype = Bug AND priority in (Highest, High)
AND created >= -7d

project = PORTAL AND sprint in openSprints()

status changed FROM "In Progress" TO "Done"
AFTER startOfMonth()
```

Reports in Jira

- **Burndown chart** — sprint progress vs ideal line.
- **Burnup chart** — completed work vs scope over time.
- **Velocity chart** — story points completed per sprint.
- **Cumulative Flow Diagram** — work in each status over time.
- **Control Chart** — cycle time analysis.
- **Sprint Report** — what was completed/added/removed in a sprint.

Boards

- **Scrum Board:** Sprints, backlog, story points, sprint goal.
- **Kanban Board:** Continuous flow, no fixed sprints, WIP limits.

How testers use Jira day-to-day

- Read user stories & acceptance criteria
- Write test cases (linked via Xray/Zephyr)
- Log defects with full details & screenshots
- Re-test fixed defects and update status
- Track sprint progress on the board
- Generate test execution reports for stakeholders

21. Agile / Scrum Basics for QA

Agile Manifesto - 4 Values

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Scrum Roles

- **Product Owner:** Owns the product backlog and priorities.
- **Scrum Master:** Facilitates the process and removes blockers.
- **Development Team:** Cross-functional (dev + QA + UX) members who deliver the increment.

Scrum Ceremonies / Events

- **Sprint Planning:** Decide what to build in the upcoming sprint.
- **Daily Stand-up:** 15-min meeting on yesterday/today/blockers.
- **Sprint Review:** Demo of work done to stakeholders.
- **Sprint Retrospective:** Team reflects: what went well, what to improve.
- **Backlog Refinement (Grooming):** Estimate & refine upcoming stories.

Sprint Lifecycle for QA

- Participate in sprint planning & estimate testing effort
- Write test cases as soon as stories are clarified
- Pair with dev for early defect detection (shift-left)
- Run smoke + functional + regression tests during the sprint
- Demo features in sprint review
- Share QA metrics in retrospective

Story Points & Estimation

Story points are a relative measure of effort using the Fibonacci scale (1, 2, 3, 5, 8, 13, 20). Teams often use **Planning Poker** to estimate collaboratively.

22. L1 Interview Questions (Manual + Selenium + Jira)

Manual / Fundamentals

Q. What is software testing?

A. It is the process of evaluating software to detect defects and ensure it meets specified requirements.

Q. Difference between QA, QC and Testing?

A. QA is process-oriented and proactive; QC is product-oriented and reactive; Testing is the actual execution of test cases to find defects.

Q. What is the difference between verification and validation?

A. Verification: Are we building the product right? (static, reviews); Validation: Are we building the right product? (dynamic, execution).

Q. Severity vs Priority?

A. Severity is the technical impact of a defect (decided by QA); Priority is the urgency to fix it (decided by business).

Q. What is a test scenario vs test case?

A. Test scenario is what to test (high-level). Test case is detailed steps with expected results.

Q. What is the bug life cycle?

A. New -> Assigned -> Open -> Fixed -> Retest -> Verified -> Closed; with states like Reopened, Rejected, Deferred, Duplicate possible in between.

Q. Define entry and exit criteria.

A. Entry criteria define conditions to begin a phase; Exit criteria define conditions to end it.

Q. What is the difference between Functional and Non-functional testing?

A. Functional checks what the system does (login works); Non-functional checks how well (performance, security, usability).

Q. What is positive and negative testing?

A. Positive testing uses valid inputs to confirm expected behaviour; Negative testing uses invalid inputs to confirm graceful failure.

Q. Difference between Alpha and Beta testing?

A. Alpha is performed in-house by employees on a pre-release build; Beta is performed by selected real end users in their environment.

Q. What is a Test Plan?

A. A document that defines scope, approach, schedule, resources, risks, and entry/exit criteria for a testing project.

Q. What is a Traceability Matrix (RTM)?

A. A document mapping requirements to test cases to ensure full coverage and impact analysis.

Q. How do you decide when to stop testing?

A. When exit criteria are met: all planned cases executed, no critical bugs open, coverage targets met, deadline reached, and stakeholder sign-off received.

Smoke / Sanity / Regression

Q. What is smoke testing?

A. A shallow, wide test executed on a new build to verify that the most critical functionalities work and the build is stable enough for deeper testing.

Q. What is sanity testing?

A. A narrow but deep test performed on a relatively stable build after minor changes to verify the fix and surrounding functionality.

Q. Smoke vs Sanity?

A. Smoke is wide and shallow on every new build; Sanity is narrow and deep on stable builds after minor changes.

Q. What is regression testing?

A. Re-running existing test cases after a code change to ensure that the change did not break any existing functionality.

Q. What is retesting vs regression?

A. Retesting verifies a single failed test after a bug fix; Regression verifies that the rest of the application still works after the change.

Q. How do you select cases for regression?

A. Critical flows, frequently failing areas, areas around the code change (impact analysis), tests for previously fixed bugs, and high-traffic user paths.

Q. Can smoke testing be automated?

A. Yes — smoke is the most common candidate for automation as it runs on every build.

Selenium L1

Q. What is Selenium?

A. An open-source suite of tools for automating web browsers, supporting many languages and browsers.

Q. Components of Selenium?

A. Selenium IDE, Selenium RC (deprecated), Selenium WebDriver and Selenium Grid.

Q. What is the difference between Selenium 3 and Selenium 4?

A. Selenium 4 uses the W3C WebDriver standard (no JSON wire), provides relative locators, an upgraded Grid, better Chrome DevTools integration, and improved APIs.

Q. What is the difference between `driver.close()` and `driver.quit()`?

A. `close()` closes only the current browser window; `quit()` closes all browser windows opened by the driver and ends the session.

Q. What are different locators in Selenium?

A. id, name, className, tagName, linkText, partialLinkText, css selector, xpath. Selenium 4 also adds relative locators.

Q. Difference between findElement and findElements?

A. findElement returns a single element (throws NoSuchElementException if not found); findElements returns a List, empty if no match.

Q. Implicit vs Explicit wait?

A. Implicit wait is set globally for all findElement calls. Explicit wait uses WebDriverWait + ExpectedConditions for a specific element/condition.

Q. What is Fluent Wait?

A. A type of explicit wait where you set total timeout, polling interval, and exceptions to ignore.

Q. How do you handle alerts in Selenium?

A. Use driver.switchTo().alert() then accept(), dismiss(), getText(), or sendKeys().

Q. How do you handle frames?

A. driver.switchTo().frame(index | name | WebElement). Use defaultContent() to go back to main page.

Q. How do you handle multiple windows?

A. Use getWindowHandle() to store parent and getWindowHandles() to get all, then switchTo().window(handle).

Q. How do you handle dropdowns?

A. For native tag, use the Select class with selectByVisibleText/Value/Index. For custom dropdowns, click to open and choose the option via locator.

Q. How do you perform mouse hover?

A. Use the Actions class: new Actions(driver).moveToElement(elem).perform().

Q. What are common exceptions in Selenium?

A. NoSuchElementException, ElementNotInteractableException, StaleElementReferenceException, TimeoutException, NoAlertPresentException, NoSuchWindowException, NoSuchFrameException, SessionNotCreatedException.

Q. What is POM?

A. Page Object Model - a design pattern where every page has a separate class containing locators and methods, making tests cleaner and more maintainable.

Q. What is PageFactory?

A. An inbuilt Selenium class that uses @FindBy annotations to initialise page elements lazily.

Q. What is TestNG?

A. A test framework for Java with annotations, parallel execution, data providers, reports, and groups.

Q. How do you achieve parallel execution in TestNG?

A. Set parallel='tests' (or methods/classes) with thread-count in testng.xml.

Jira L1

Q. What is Jira?

A. An Atlassian tool for project management, agile boards, issue tracking, and test management.

Q. What is an Epic, Story, and Bug?

A. Epic = large work item across sprints; Story = single feature from user perspective; Bug = a defect.

Q. What is JQL?

A. Jira Query Language — a structured language to search for issues based on fields and conditions.

Q. How is a bug logged in Jira?

A. Click Create -> select project -> Bug issue type -> fill summary, description with reproduction steps, expected/actual, severity, priority, environment, screenshot -> assign -> save.

Q. What is the difference between a Scrum board and Kanban board?

A. Scrum board uses fixed-length sprints with planning & commitments; Kanban board is continuous flow with WIP limits and no sprints.

Q. What is a Sprint in Jira?

A. A time-boxed period (usually 2 weeks) where a team commits to and delivers a set of items.

Q. Have you used any Jira plugin for test management?

A. Yes - Xray, Zephyr (Squad/Scale), TestRail integration are common.

Q. What is a burndown chart?

A. A chart showing remaining work in a sprint vs ideal line, used to track sprint progress.

23. L2 Interview Questions (Scenario / Coding)

L2 questions test depth, real-world thinking, and coding skills. Practise speaking your thought process out loud.

Scenario & Strategy

Q. How would you test a login page?

A. Functional: valid/invalid credentials, empty fields, max length, special characters, SQL injection. Negative: locked accounts, expired passwords. UI: alignment, labels, tab order, password masking. Security: brute force, HTTPS, password hashing. Compatibility: browsers, devices. Performance: response time under load. Accessibility: keyboard navigation, screen reader.

Q. How would you test a search functionality?

A. Empty search, single character, exact match, partial match, special characters, very long strings, pagination, sorting/filtering, response time, suggestions/autocomplete, accent/case sensitivity, search history, and cross-locale behaviour.

Q. You have only 2 hours before release. How will you test?

A. Run automated smoke + critical regression first. Then run risk-based exploratory testing on changed and high-impact areas. Communicate coverage and any open risks to the team for go/no-go.

Q. A bug appears in production but not in QA. What do you do?

A. Compare environments (data, config, browser, network); reproduce on prod-like data; capture logs and screenshots; raise a P1 incident; add the missing scenario to regression suite; do a 5-Why root cause analysis with the team.

Q. Developer says 'works on my machine' but you reproduce the bug. How do you handle it?

A. Provide exact steps, build number, environment details, video/screenshot, and logs. Offer to demo together. Stay objective: the goal is fixing it, not winning.

Q. How would you ensure that your regression suite stays effective?

A. Periodically review and remove obsolete tests; add tests for new features and fixed defects; group tests by priority; analyse flaky tests; track which tests catch real defects (test effectiveness metric).

Q. What metrics do you track as a QA engineer?

A. Test coverage %, defect density, defect leakage, defect rejection ratio, automation coverage, pass/fail rate, mean time to detect/repair, escaped defects, and sprint velocity for QA tasks.

Q. How do you handle flaky tests in Selenium?

A. Identify them via repeated runs; investigate root cause (waits, sync, data, environment); fix with proper explicit waits, stable locators, isolated test data, and retries; quarantine the test if it cannot be stabilised quickly.

Q. How would you test an API?

A. Validate status code, response time, response body schema, headers, authentication/authorization, positive/negative payloads, boundary values, security (e.g., missing token), rate limiting, and idempotency. Tools: Postman, REST Assured, JMeter.

Q. What is shift-left testing?

A. Moving testing activities earlier in the SDLC: requirement reviews, static analysis, unit test focus, TDD/BDD, early test design and pairing with developers to catch defects sooner.

Selenium - Advanced & Coding

Q. How do you handle `StaleElementReferenceException`?

A. Re-locate the element after a DOM change, wrap interactions in retry logic, avoid storing `WebElement` references across page changes, and prefer fresh `findElement` calls.

Q. How do you take a screenshot only on failure with TestNG?

A. Implement `ITestListener.onTestFailure()`, inside which cast the driver to `TakesScreenshot` and save the image to a reports folder.

Q. How do you read configuration from properties file?

A. Use `java.util.Properties`: load the `.properties` file with `FileInputStream`, then `prop.getProperty("url")`.

Q. How would you handle dynamic XPath?

A. Use `contains()`, `starts-with()`, `normalize-space()`, `text()`, or relative XPath built around stable anchor elements (parent, sibling). Avoid absolute paths and indices when possible.

Q. How do you scroll to an element?

A. `((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView(true);", element);` or `window.scrollTo(0, 500)`.

Q. How to perform parallel execution with TestNG and Selenium?

A. Set `parallel='methods'` (or `'classes'/'tests'`) and `thread-count` in `testng.xml`; ensure `WebDriver` is thread-safe using `ThreadLocal` in `BaseTest`.

Q. Why use `ThreadLocal`?

A. To give each parallel test thread its own driver instance and avoid cross-thread `WebDriver` corruption.

Q. How do you achieve cross-browser testing?

A. Drive the browser type from a parameter (`testng.xml` or config), use a `DriverFactory` that returns the right `WebDriver`, and run on Selenium Grid or cloud platforms like BrowserStack/Sauce Labs.

Q. How do you integrate Selenium tests with Jenkins?

A. Create a Jenkins job pointing to the Maven project, configure Maven goals (`mvn clean test`), publish TestNG/Allure reports as post-build steps, and schedule via cron or trigger on every Git commit using webhook.

Common coding tasks

Find broken links on a page

```
List<WebElement> links = driver.findElements(By.tagName("a"));
for (WebElement link : links) {
    String url = link.getAttribute("href");
    if (url == null || url.isEmpty()) continue;
    HttpURLConnection conn = (HttpURLConnection)
```

```

        new URL(url).openConnection();
    conn.setRequestMethod("HEAD");
    conn.connect();
    if (conn.getResponseCode() >= 400) {
        System.out.println("Broken: " + url + " -> "
            + conn.getResponseCode());
    }
}

```

Verify an element is enabled and displayed

```

WebElement btn = driver.findElement(By.id("save"));
Assert.assertTrue(btn.isDisplayed());
Assert.assertTrue(btn.isEnabled());

```

Login Util using POM + DataProvider

```

@Test(dataProvider="creds")
public void loginTest(String u, String p, String expected) {
    LoginPage login = new LoginPage(driver);
    login.loginAs(u, p);
    Assert.assertEquals(login.getMessage(), expected);
}

@DataProvider(name="creds")
public Object[][] creds() {
    return new Object[][] {
        {"u1", "p1", "Welcome"},
        {"u2", "wrong", "Invalid"},
        {"", "", "Required"}
    };
}

```

24. Bonus: Resume Tips & Final Checklist

Resume Tips for Fresher QA / SDET

- Keep it to one page. Recruiters spend < 30 seconds on first pass.
- Lead with a short objective: "Aspiring QA engineer with hands-on Selenium and Java skills, eager to contribute to robust test automation."
- Skills section: Java, Selenium WebDriver, TestNG, Maven, Git, Jenkins, Jira, SQL, REST Assured, Postman.
- Project section: describe 1-2 automation projects with stack, role, and impact. e.g., "Built POM + TestNG framework for an e-commerce site; automated 120+ test cases; reduced manual effort by 60%."
- Add link to your GitHub with the code — this drastically improves shortlisting.
- Mention certifications: ISTQB Foundation Level, Selenium WebDriver course, etc.
- Use action verbs: built, designed, automated, executed, integrated.

Final Interview Checklist

- Be crystal clear on STLC, bug life cycle, severity vs priority, and smoke vs sanity vs regression.
- Memorise the differences between locators, waits, and common exceptions.
- Be able to write a small Selenium login automation on a whiteboard or shared editor.
- Practise explaining a framework you have built — modules, file structure, libraries, reports.
- Know JQL basics and how you log defects in Jira.
- Brush up on SQL: SELECT, JOIN, WHERE, GROUP BY, HAVING, subquery.
- Have 2 STAR (Situation-Task-Action-Result) stories ready: one about teamwork, one about a tough bug.
- Smile, listen carefully, and think out loud during scenario questions.

Suggested 30-Day Study Plan

Week	Focus
Week 1	Manual testing fundamentals, SDLC/STLC, test design techniques, bug life cycle, severity vs priority.
Week 2	Smoke, sanity, regression, API testing basics, SQL queries. Practise writing test cases.
Week 3	Selenium WebDriver, locators, waits, alerts, frames, windows, Actions, TestNG.
Week 4	POM + PageFactory, data-driven framework, mini project on GitHub, Jenkins integration, mock interview

Closing Note

Confidence comes from clarity, and clarity comes from practice. Read each chapter, attempt the interview questions out loud, and try writing the Selenium snippets in your own IDE. Treat every mock interview as a real one. All the best for your L1 and L2 interviews! ■